

Capture the Flag 5: Make the System Interactive

Goal: Transform your Student Directory into an interactive application where administrators can favorite students, edit information, and remove records using Flutter interactions and state management.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

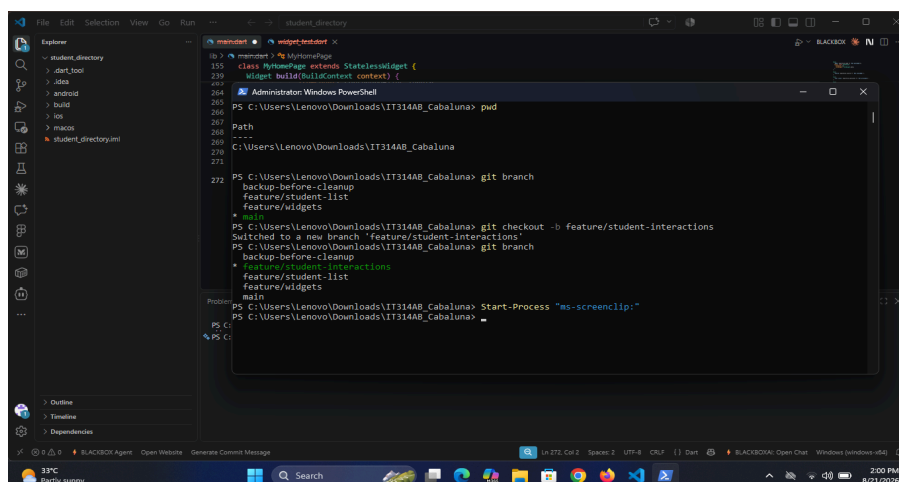
🚩 Flag 0 — Create Your Feature Branch

Objective: Create a new feature branch before adding interactions.

Create:

feature/student-interactions

Screenshot:



The screenshot shows a Windows terminal window with a dark background. The terminal output is as follows:

```
PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna> pwd
C:\Users\Lenovo\Downloads\IT314AB_Cabaluna

PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna> git branch
backup-before-cleanup
feature/student-list
feature/widgets
main
PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna> git checkout -b feature/student-interactions
Switched to a new branch 'feature/student-interactions'
PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna> git branch
backup-before-cleanup
feature/student-interactions
feature/student-list
feature/widgets
main
PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna> Start-Process "ms-screenclip;"
PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna>
```

The background of the terminal shows a file explorer with a project structure including folders like 'student_directory', 'dart_tool', 'ios', 'macos', and 'student_directory.iml'.

- Terminal showing your current branch

🚩 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using **onPressed**.

Right now your buttons are only decorations.

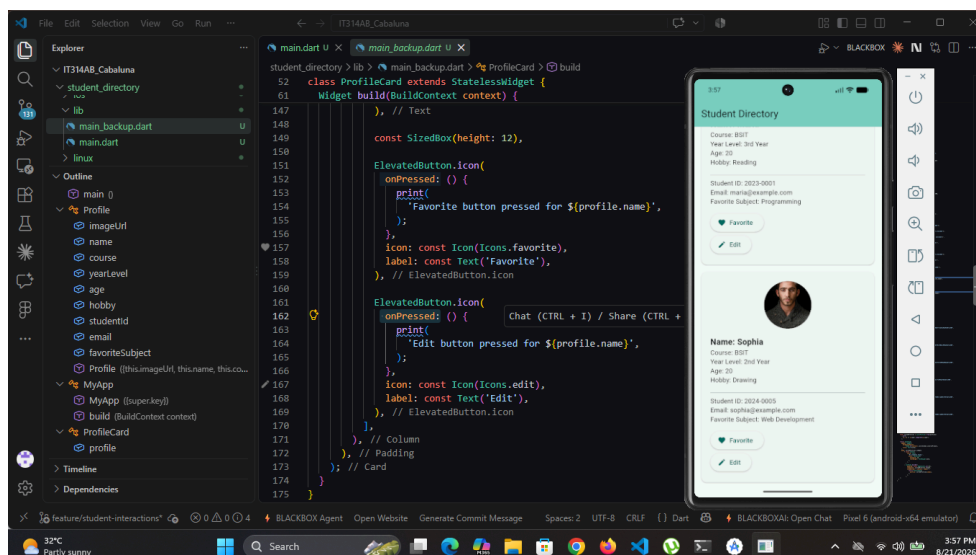
Research how Flutter's **onPressed** works.

The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

Test

When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

Screenshot:



- Student Card with buttons
- **onPressed** inside your code
- Button reacting or terminal message

🚩 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using `onTap`.

Buttons aren't the only interactive elements.

Research Flutter's `onTap`.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

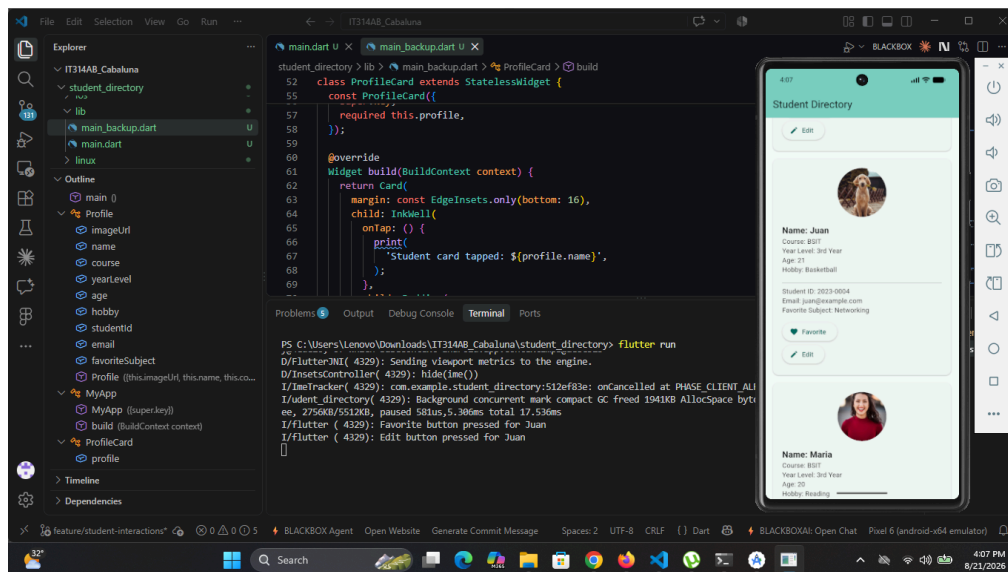
Test

The user should be able to:

- Tap the Favorite button
- Tap the Edit button
- Tap anywhere else on the Student Card

Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

Screenshot:



- `onTap` in your code
- Student Card being tapped

🚩 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

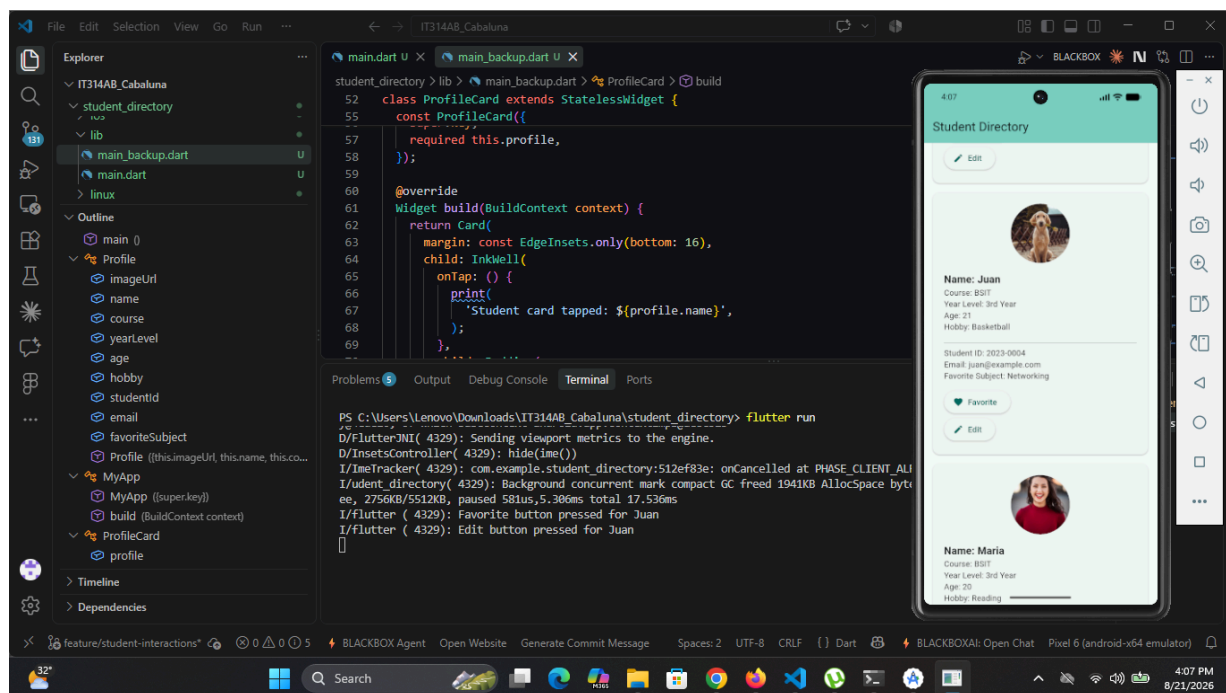
The important part is understanding that `setState` tells Flutter to rebuild the UI.

Test

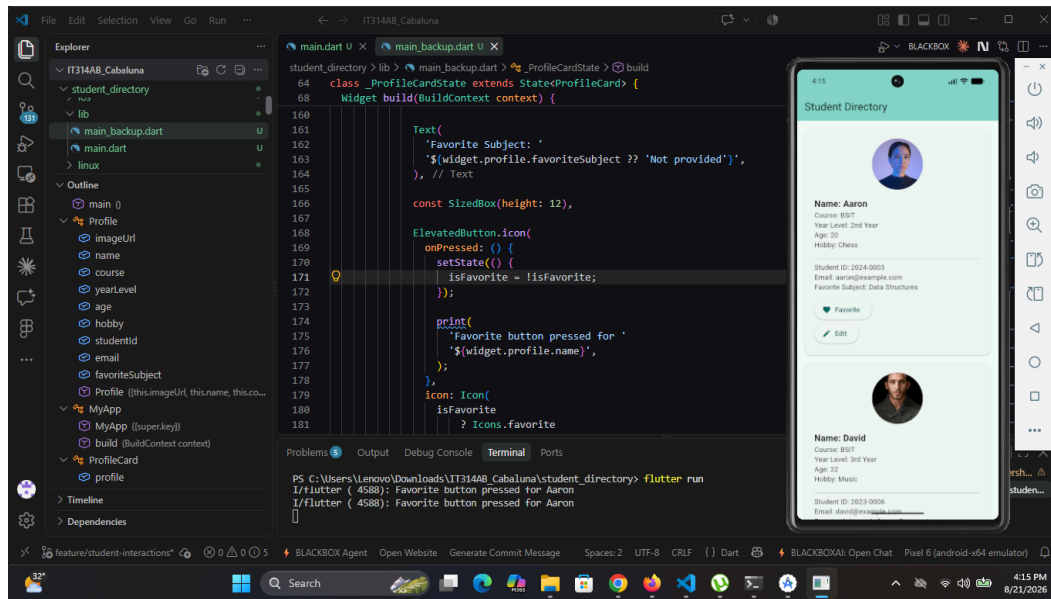
The screen should visibly change immediately after pressing the button.

Screenshot:

BEFORE



AFTER



- `setState` in your code
- Before and after interaction

Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

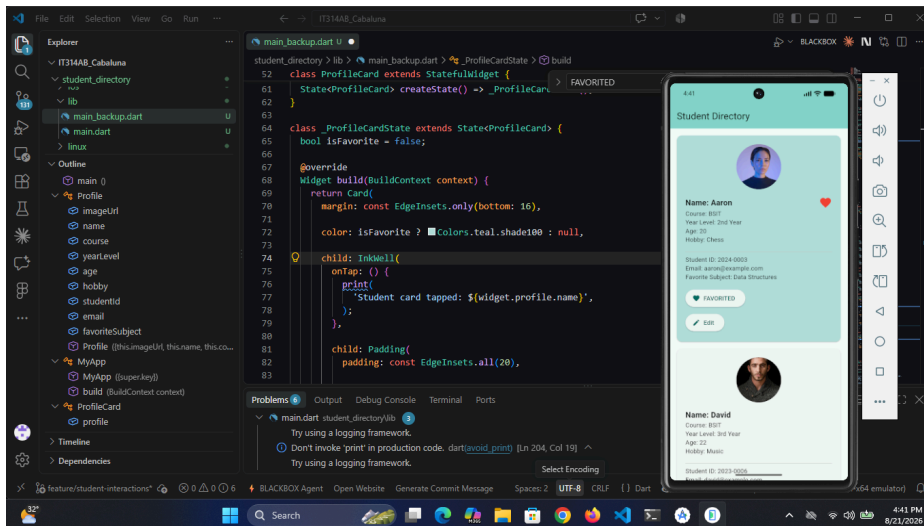
Each student's favorite status should work independently.

Test

- Favorite one student.
- Other students should remain unchanged.
- Favorite another student.

Both should keep their own state.

Screenshot:



- Favorited student
- Non-favorited student
- Favorite icon changing

🚩 Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart [List](#).

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

Test

Before:

- Student A
- Student B
- Student C

After deleting Student B:

- Student A
- Student C

No empty spaces should remain.

Screenshot:

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

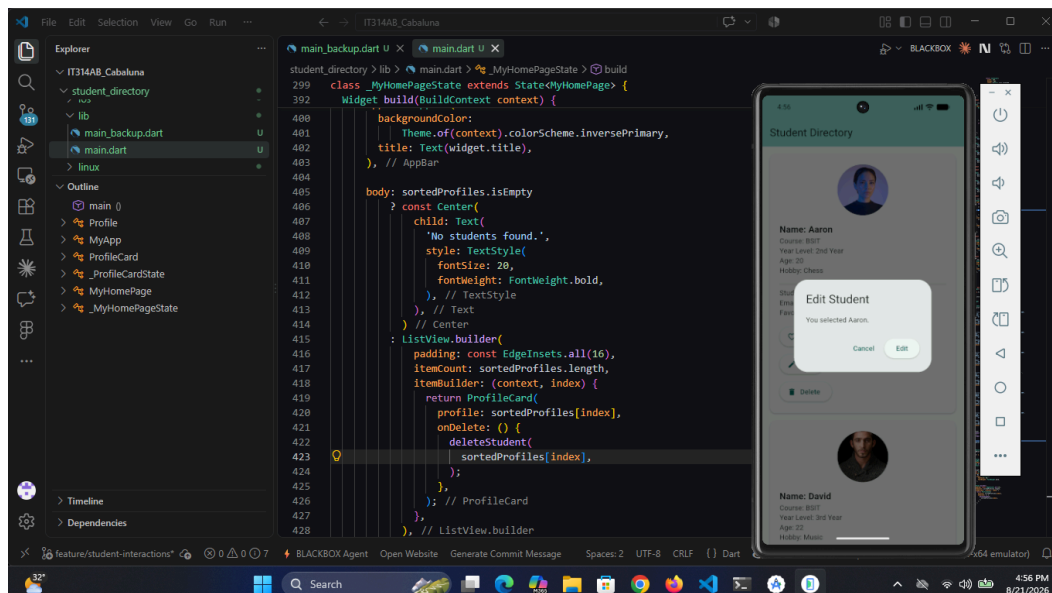
- `AlertDialog`
- `SnackBar`
- Bottom Sheet

The goal is simply to launch an edit action, not build the complete editor yet.

Test

Pressing Edit should open your chosen interface.

Screenshot:



- Edit button
- Opened dialog, snackbar, or bottom sheet

Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

This is your final interaction challenge.

Verify that every student behaves independently.

Example scenario:

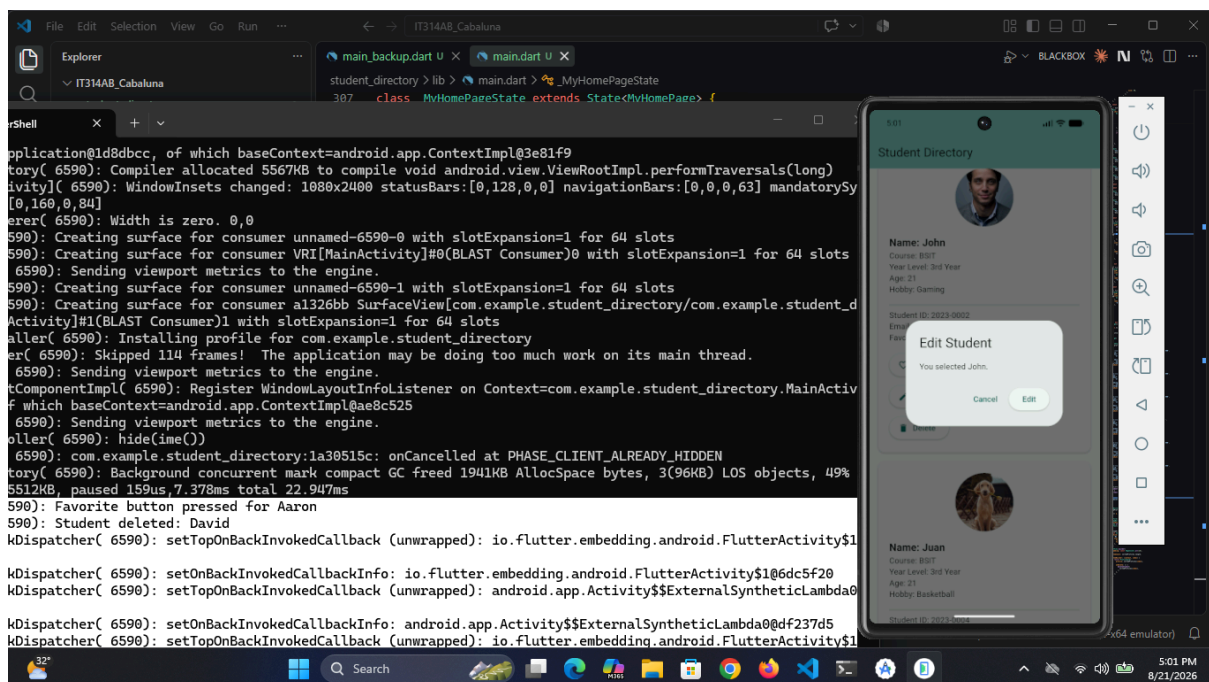
- Favorite Student A.
- Delete Student C.
- Open Edit for Student B.

Your application should continue functioning correctly without affecting unrelated students.

Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

Screenshot:



- Multiple interactions working together
- Application still functioning normally

Flag 8 — Save Your Progress

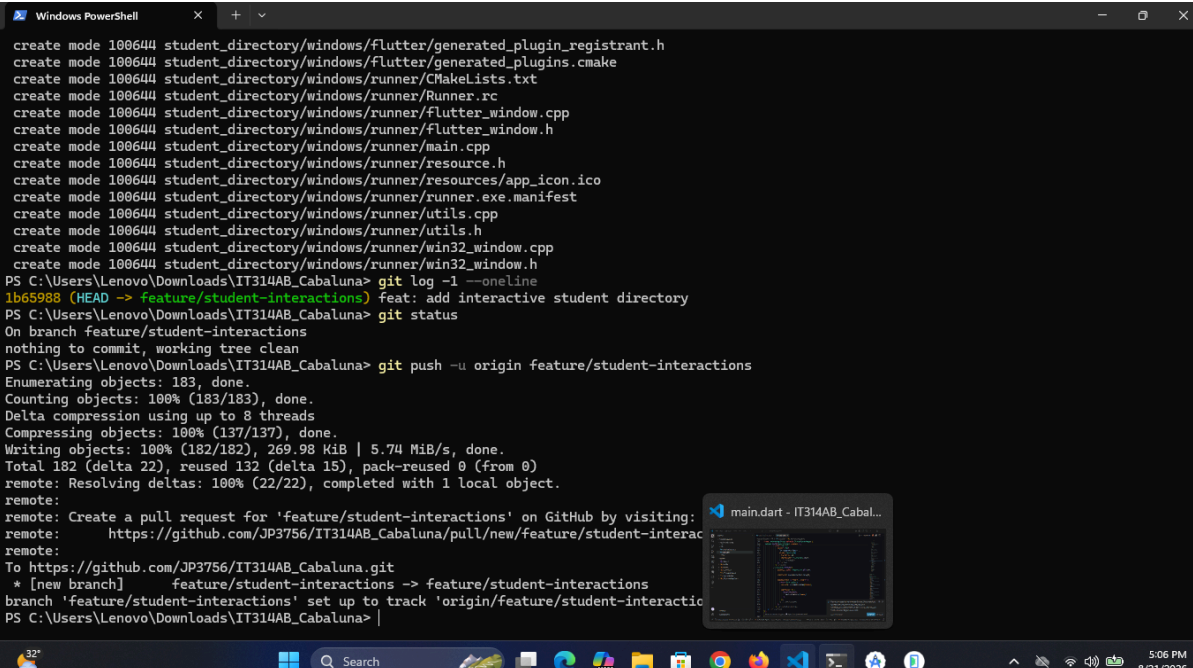
Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

Branch:

feature/student-interactions

Screenshot:



```
create mode 100644 student_directory/windows/Flutter/generated_plugin_registrant.h
create mode 100644 student_directory/windows/Flutter/generated_plugins.cmake
create mode 100644 student_directory/windows/runner/CMakeLists.txt
create mode 100644 student_directory/windows/runner/Runner.rc
create mode 100644 student_directory/windows/runner/flutter_window.cpp
create mode 100644 student_directory/windows/runner/flutter_window.h
create mode 100644 student_directory/windows/runner/main.cpp
create mode 100644 student_directory/windows/runner/resource.h
create mode 100644 student_directory/windows/runner/resources/app_icon.ico
create mode 100644 student_directory/windows/runner/runner.exe.manifest
create mode 100644 student_directory/windows/runner/utils.cpp
create mode 100644 student_directory/windows/runner/utils.h
create mode 100644 student_directory/windows/runner/win32_window.cpp
create mode 100644 student_directory/windows/runner/win32_window.h
PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna> git log -1 --oneline
1b65988 (HEAD -> feature/student-interactions) feat: add interactive student directory
PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna> git status
On branch feature/student-interactions
nothing to commit, working tree clean
PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna> git push -u origin feature/student-interactions
Enumerating objects: 183, done.
Counting objects: 100% (183/183), done.
Delta compression using up to 8 threads
Compressing objects: 100% (137/137), done.
Writing objects: 100% (182/182), 269.98 KiB | 5.74 MiB/s, done.
Total 182 (delta 22), reused 132 (delta 15), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (22/22), completed with 1 local object.
remote:
remote: Create a pull request for 'feature/student-interactions' on GitHub by visiting:
remote:   https://github.com/JP3756/IT314AB_Cabaluna/pull/new/feature/student-interactions
remote:
To https://github.com/JP3756/IT314AB_Cabaluna.git
 * [new branch]      feature/student-interactions -> feature/student-interactions
branch 'feature/student-interactions' set up to track 'origin/feature/student-interactions'
PS C:\Users\Lenovo\Downloads\IT314AB_Cabaluna> |
```

- Successful commit
- Current branch
- Terminal showing the commit